Software Testing and Quality Assurance

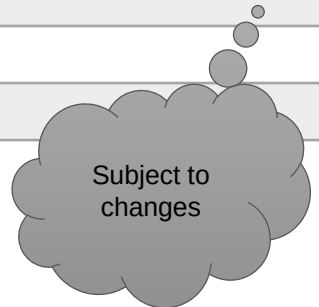
Lecture 6

Fabian Fagerholm

fabian.fagerholm@aalto.fi

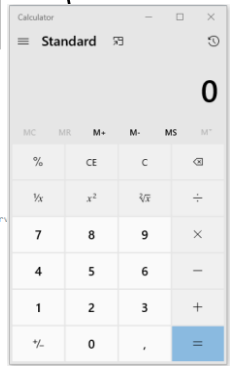
Aalto University
School of Science

Week	Lecture	Topic	Assignment deadlines (Tuesdays 10:00 unless otherwise specified)
36	3.9.2024	Introduction and practicalities	
37	10.9.2024	Software quality	Individual assignments 1
38	17.9.2024	Software testing: levels, test case analysis and design	Group registration (DL: 20.9.)
39	24.9.2024	Testing techniques: Black-box, white-box testing	Individual assignments 2, Group assignment 1
40	1.10.2024	Testing techniques: Manual testing	Individual assignments 3
41	8.10.2024	Testing techniques: Black-box, white-box, manual testing	Individual assignments 4
42	15.10.2024	(No lecture)	
43	22.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 5
44	29.10.2024	Testing techniques: Static code analysis and software metrics	Individual assignments 6, Group assignment 2
45	5.11.2024	Guest lecture	Individual assignments 7
46	12.11.2024	Continuous Integration and Continuous Delivery/Deployment, Test management, and Course summary	Individual assignments 8
47	19.11.2024	(No lecture)	Individual assignments 9
48	26.11.2024	(No lecture)	Individual assignments 10, Group assignment 3



Concepts

System Under Test (SUT): The software system being tested

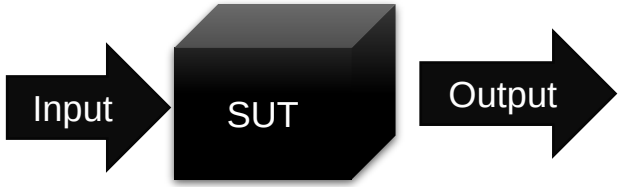



```
// Copyright (c) Microsoft Corporation. All rights reserv
// Licensed under the MIT License.

using CalculatorApp.ViewModel.Common;
using System;

namespace CalculatorApp
{
    namespace Converters
    {
        /// <summary>
        /// Value converter that translates true to false and vice versa.
        /// </summary>
        [Windows.Foundation.Metadata.WebHostHidden]
        public sealed class RadixToStringConverter : Windows.UI.Xaml.Data.IValueConverter
        {
            public object Convert(object value, Type targetType, object parameter, string language)
            {
```

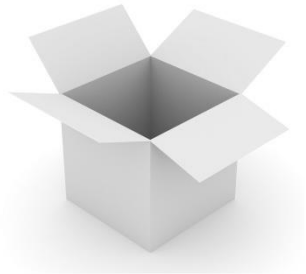
Aalto University
School of Science



Black-box testing: Observing SUT outputs for specified inputs *without looking at the internals*

Coverage: How much of the source code has been tested?

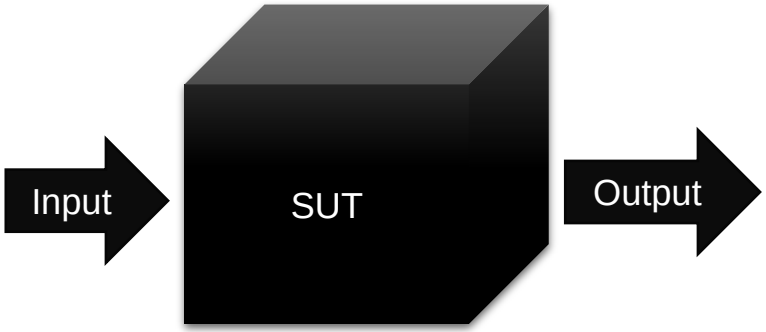
Grey-box testing: Black-box + White-box



White-box testing: Test SUT behavior *using knowledge of its internal structure*
(Also: clear-box, glass-box, or *structural testing)*

Black-box testing

- Testing that does not examine the internals of the SUT
  - Inputs
  - Outputs
- Also called *functional testing, behavioural testing*, or *specification-based testing*
- Can be applied on all levels of testing (unit, integration, system, acceptance)
- Automated tests can be black-box tests
- Exploratory testing is a kind of black-box testing
- Advantage: can be efficient and effective
- Disadvantage: can never be sure of how much of the system has been exercised
- **Emphasis on finding a limited set of test cases (inputs) that test the system as thoroughly as possible**



Some very specific inputs could lead to a rare execution path that cannot be known without seeing the code

```
if (student.number == 1234567) {
    student.grade = 5;
}
```

Techniques for designing black-box tests

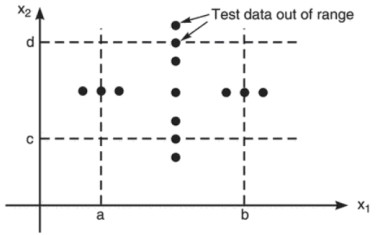
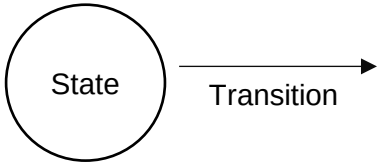


FIGURE 4.2 Robustness Test Cases.

Chopra (2018)

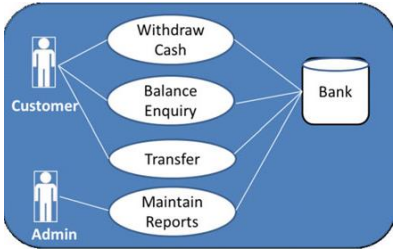
Boundary value analysis
Robustness testing



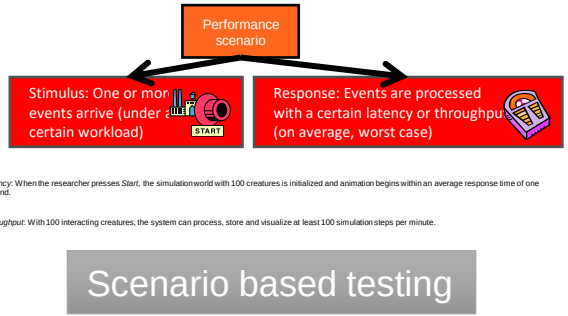
State transition testing

	Rule 1	Rule 2	Rule 3	Rule 4
Input conditions				
Customer with loyalty card?	Yes	Yes	No	No
Spend > 200?	Yes	No	Yes	No
Actions				
Discount	10%	5%	2%	0%
Accumulate loyalty points?	Yes	Yes	No	No
	Test case 1	Test case 2	Test case 3	Test case 4

Decision table testing



Use case based testing



Scenario based testing

White-box testing

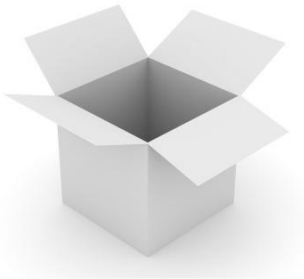
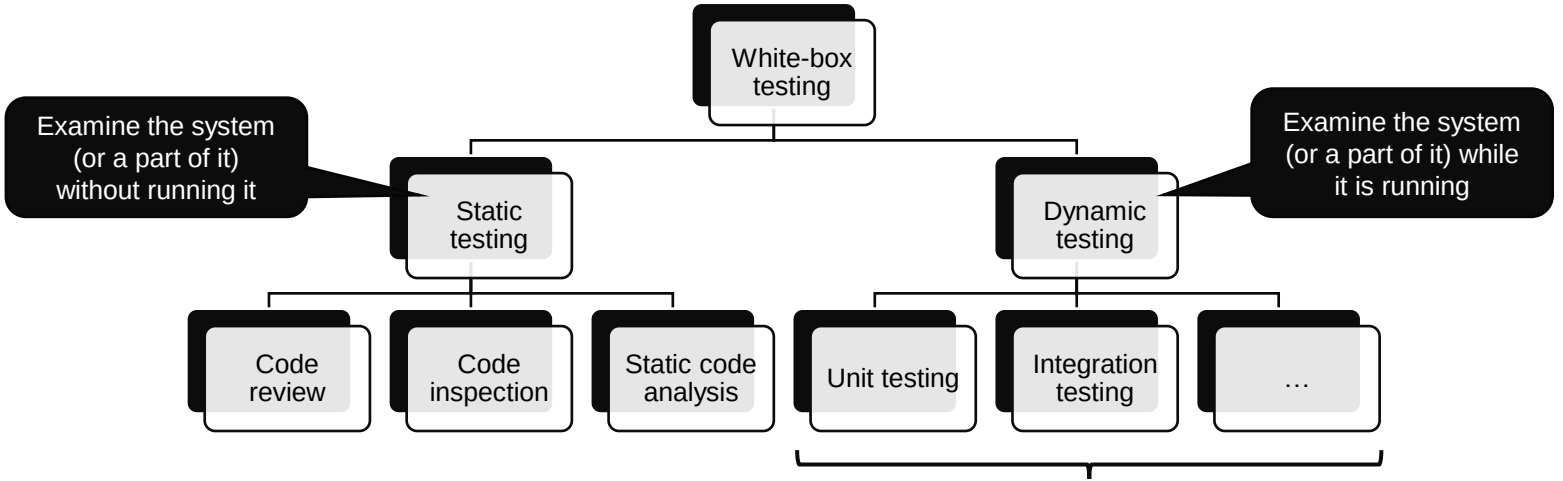


Photo CC BY-SA-NC

- Also: clear-box, glass-box, or *structural testing*
- Designing tests for the SUT *using knowledge of its internal structure*
- *Today: techniques for designing test cases based on a white-box approach*

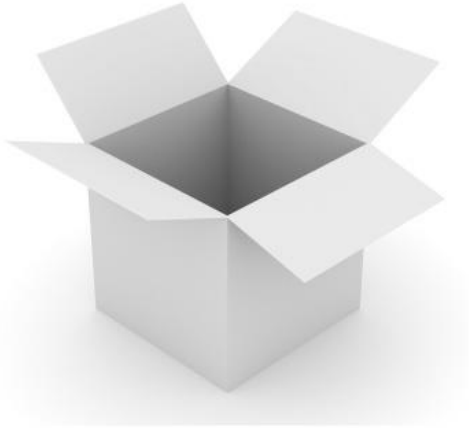


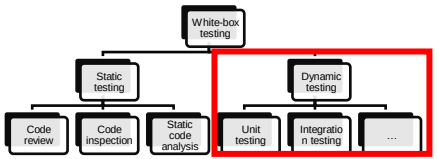
Potentially all levels of the testing pyramid – just like in black-box testing but based on knowledge of the internal structure

Dynamic white-box testing

- Possible applications on different testing levels
  - Unit: Test paths within individual modules before integratior
  - Integration: Test paths between modules (also: data flow)
  - System: End-to-end application flow from user perspective
- Challenges
  - Cannot test all possible paths
  - Test cases may not detect data sensitivity errors (e.g., p=q/r may execute correctly except when r=0)
  - Assumes control flow is correct (non-existent paths cannot be discovered)
  - The tester must have programming skills to evaluate the SUT

Dynamic white-box testing emphasises paths – test case design is based on path analysis





Manual testing

Photo: CC BY-SA

- “Human-present testing”
  - A human steers the testing and performs some essential testing actions
  - A human makes the judgement of whether the test passes or not
  - Some tools could be used to help
- Problems can arise with unsystematic manual testing:
  - Ad-hoc testing
  - Manual testing can be too slow
  - Not repeatable
  - Not reproducible
  - Not transferable

Systematic manual testing

Scripted testing

Exploratory testing

Scripted testing

- The tester follows a script to perform tests
- **Rigid scripts**
- Flexible scripts
- Scripted testing can be as rigid or as flexible as necessary

BUT 1: Very rigid scripts could be automated

BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

Functional area	Test Name	Test Steps	Test Data	Expected Results
Sign-up	Sign-in success	1. Enter a valid username and password 2. Click Login	Username: mary Password: Secret1$	User is logged in successfully
Sign-up	Sign-in with incorrect password	1. Enter valid username 2. Enter invalid password 3. Click Login	Username: mary Password: wrong0	User is returned to login page and wrong username or password message is shown

Scripted testing

- The tester follows a script to perform tests
- Rigid scripts
- **Flexible scripts**
- Scripted testing can be as rigid or as flexible as necessary

BUT 1: Very rigid scripts could be automated

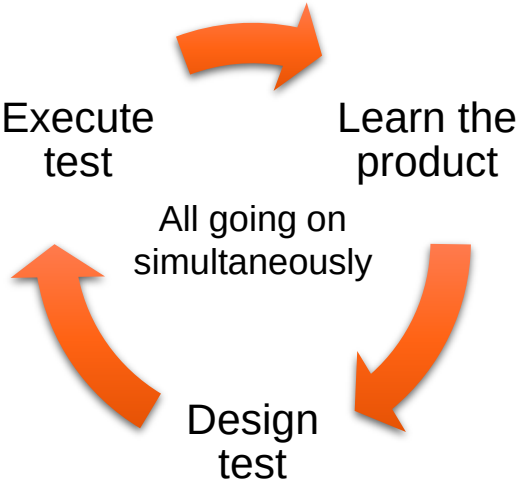
BUT 2: For very flexible scripts to work, testers need specific instructions on how to handle choices and uncertainty – otherwise it becomes ad-hoc testing!

Exploratory testing can be used instead

Chat bot test

1. Open the chat bot
2. Interact with the chat bot interface
3. Report findings

Exploratory testing



- No test scripts – but can use guiding plan
- Test documentation (cases, results) recorded as the test proceeds
- Tools can be used (debugger, screen recorder, etc.)
- Focus on *varying things* to explore the SUT
- Systematic guidance increases the chances of finding faults
- Requires skill and experience
- Management challenges
    - Keeping track of testing progress (especially with multiple testers)
    - Summarising the state and results of testing (e.g., for internal and external stakeholders)

Approaches to exploratory testing

- Exploratory testing in the small and in the large
  - Small decisions: the detailed choices the tester makes when exploring
  - Large decisions: choices about managing the overall exploration

Session-based test management (Bach, 2000)

- Strictly time-boxed sessions of free exploration
- Management on the level of sessions
- Sessions are directed and planned

Tour-based exploratory testing (Whittaker, 2009)

- Tourist exploration as a metaphor for testing
- Management on the level of tours
- Tours have a theme or focus to guide tester activities

Bach, J. (2000). Session-Based Test Management. Software Testing and Quality Engineering, 2(6).
Whittaker, J. A. (2009). Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design. Addison-Wesley Professional.

Examples of ”small” choices

User input

State

Code paths

User data

Environment

Whittaker, J. A. (2009). Exploratory Software Testing: Tips, Tricks, Tours, and Techniques to Guide Test Design. Addison-Wesley Professional.

Discussion

Where should software testing start?

(How do we know where to start looking for defects?)

Where to start testing?

Start from requirements

Start from a model of typical defects

Quality characterisation model

(based on standards, tailored to the organisation)

Functionality | Reliability | Usability

Efficiency | Maintainability | Portability

Decomposition

Metrics

Selection, tailoring

Project-specific quality model

Project-specific quality requirements and attributes

Assure

Project-specific metrics and thresholds

Selection, tailoring

Abstract understanding of quality: conformance to requirements

High-level require-ments | High-level require-ments | High-level require-ments

Satisfy

Detailed specifi-cations | Detailed specifi-cations | Detailed specifi-cations | Detailed specifi-cations

Software Quality Management (SQM)

Software Quality Assurance (SQA)	Software Quality Control (SQC)	Software Quality Planning (SQP)	Software Process Improvement (SPI)
• **A general quality guide, not specific to a project**	• **Examine artefacts for compliance** • E.g. inspection, reviews, testing	• **Project-level quality commitment** • **Based on SQA**	• Improve process quality

Examine for compliance

Aalto University
School of Science

Defect taxonomies

A classification of typical types of defects

- Generate ideas for test design
- Audit test plans to determine coverage
- Understand defects, their types, and severities
- Understand the process producing the defects
- Improve the development process
- Improve the testing process
- Train new testers about areas that need testing
- Explain the complexities of software testing to management

Project-level defect taxonomy example

Class	Element	Attribute
Product engineering	Requirements	Stability Completeness Clarity Validity Feasibility Precedent Scale Functionality
	Design	Functionality Difficulty Interfaces Performance Testability
	Code and Unit Test	Feasibility Testing Coding/implementation
	Integration and Test	Environment Product System
…	…	…

Concern: Incomplete requirements
→
Emphasis: Exploratory testing

Concern: Imprecise requirements
→
Emphasis: Decision tables, state transition diagrams

Concern: System performance
→
Emphasis: Performance testing

Carnegie-Mellon University Software Engineering Institute – Taxonomy-Based Risk Identification

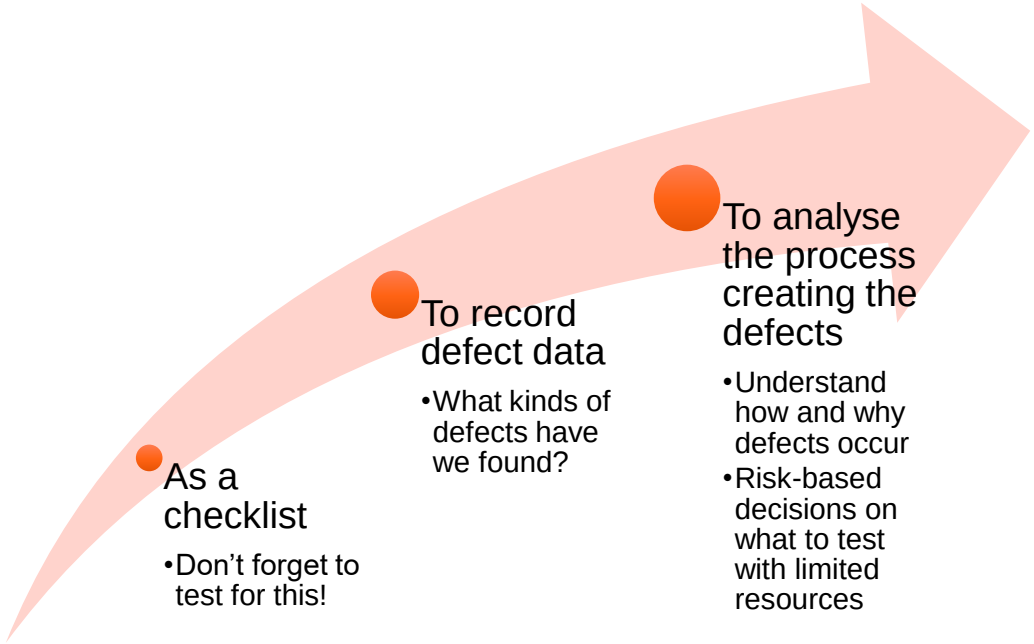
Beizer's taxonomy

- 4 levels (top two levels shown →)
- Detailed list of precise defect types
- Points the tester towards different kinds of defects to check for

1xxxx	**Requirements**
11xx	Requirements incorrect
12xx	Requirements logic
13xx	Requirements, completeness
14xx	Verifiability
15xx	Presentation, documentation
16xx	Requirements changes
2xxx	**Features and Functionality**
21xx	Feature/function correctness
22xx	Feature completeness
…	…

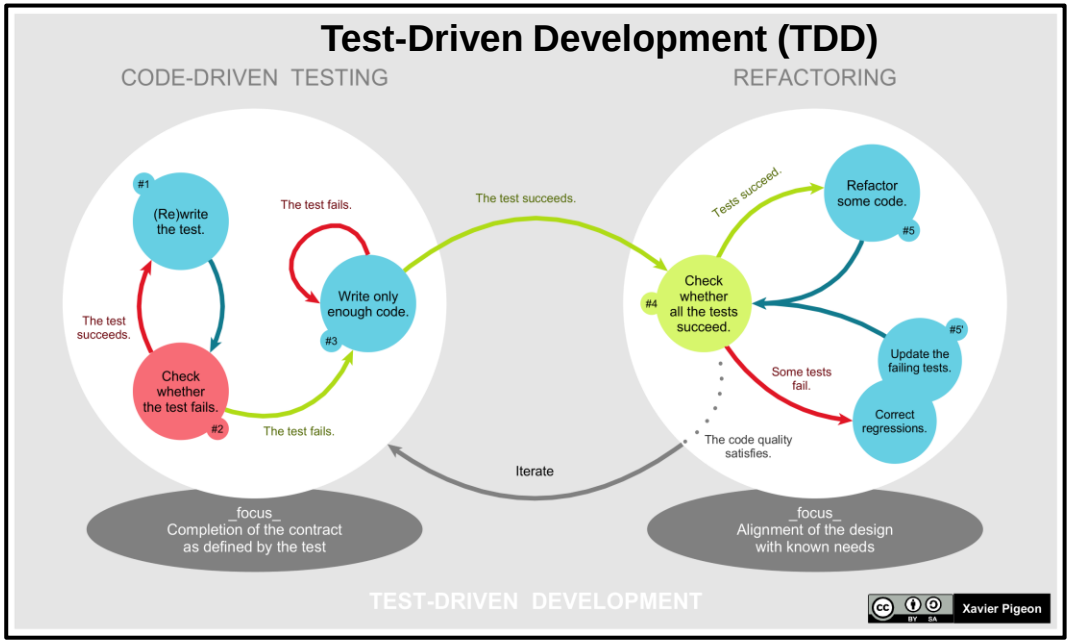
Beizer, B. (2003). Software Testing Techniques. 2nd ed. Dreamtech.

Using a defect taxonomy



Test-driven development (TDD)

- TDD is an approach where tests are written before the code is implemented
- Tries to avoid biasing the test design
  - Testers and developers tend to have *confirmation bias* – looking only for evidence that confirms assumptions (or wishes)
  - Writing the test first can promote a more critical mindset
- Strongly connected to *refactoring* to maintain code quality



Discussion

Pick 2-3 testing techniques from the course

Can you explain them to your neighbour / group members?

Which aspects are clear / unclear to you?

Try to solve the unclear parts together

Next steps

Use the related reading for more details. Reserve enough time to set up your environment.

- Individual assignments in MyCourses
  - Exploratory testing

Deadline: before next lecture (Tuesday by 10:00)

  - Note: next week is exam week – no lecture!

- Getting help
  - Ask in the course chat, see Communication section in MyCourses
  - Personal questions by email

Study smarter, not harder!

- **Plan: when and where**
- **Go deep at your own pace**
- **Get some rest**
- **Practice makes perfect**
- **Enjoy it ☺**